

Cloud Design Patterns

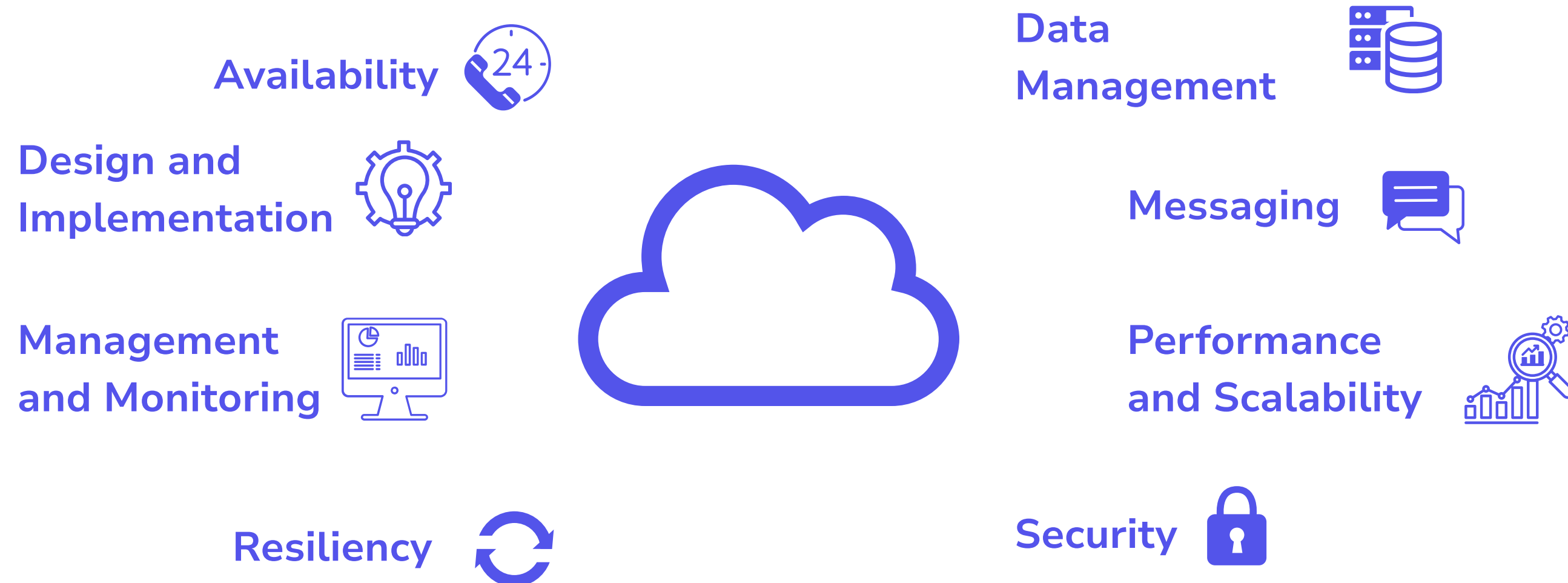
Content

- **What Are They?**
- **Priority Queue Pattern**
 - Single Queue
 - Multiples Queues
 - When to use / Avoid
 - Recommendations
- **Throttling**
 - Problems and Considerations
 - When to use this pattern
 - Workload design
 - Throttling Example



What are they?

Cloud Design Patterns are proven, tech-agnostic solutions to overcome common architectural challenges in any environment.



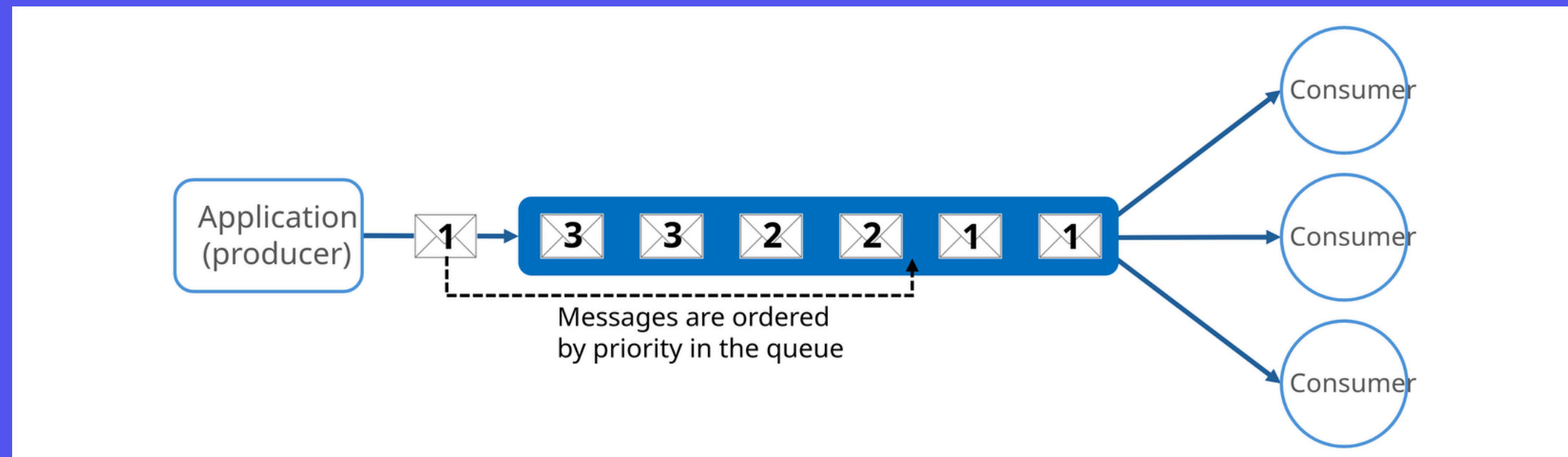
Priority Queue Pattern

The Priority Queue pattern uses message queues to process urgent tasks faster than lower-priority ones, ensuring critical service-level agreements are met.



Single Queue

In a single queue approach, the application assigns a priority to each message and sends all messages to a single queue.



<https://learn.microsoft.com/en-us/azure/architecture/patterns/priority-queue>

Single Queue Example

```
1 import heapq
2 import time
3 from dataclasses import dataclass
4 from typing import Any
5
6 @dataclass
7 class Message:
8     priority: int
9     content: Any
10
11     def __lt__(self, other):
12         return self.priority < other.priority
13
14 class SinglePriorityQueue:
15     def __init__(self):
16         self._queue = []
17
18     def send(self, priority: int, content: Any):
19         """Envía un mensaje a la cola con una prioridad específica."""
20         message = Message(priority, content)
21         heapq.heappush(self._queue, message)
22         print(f"[SEND] Mensaje '{content}' con prioridad {priority} añadido.")
23
24     def receive(self) -> Message | None:
25         """Recibe el mensaje de mayor prioridad (el que tiene el número más bajo)."""
26         if not self._queue:
27             return None
28         return heapq.heappop(self._queue)
29
30 if __name__ == "__main__":
31     queue = SinglePriorityQueue()
32     queue.send(3, "Tarea baja")
33     queue.send(1, "Tarea URGENTE")
34     queue.send(2, "Tarea normal")
35
36     print("\n--- Procesando mensajes (Single Queue) ---")
37     while True:
38         msg = queue.receive()
39         if msg is None:
40             break
41         print(f"[PROCESS] Procesando: '{msg.content}' (Prioridad {msg.priority})")
42         time.sleep(0.5)
```

```
OUTPUT  DEBUG CONSOLE  PROBLEMS  TERMINAL  PORTS

[Running] python -u "/home/noodle/Documents/Plataformas II/Priority_Queue_Pattern_Example/Single_Queue/Single.py"
[SEND] Mensaje 'Tarea baja' con prioridad 3 añadido.
[SEND] Mensaje 'Tarea URGENTE' con prioridad 1 añadido.
[SEND] Mensaje 'Tarea normal' con prioridad 2 añadido.

--- Procesando mensajes (Single Queue) ---
[PROCESS] Procesando: 'Tarea URGENTE' (Prioridad 1)
```

AWS

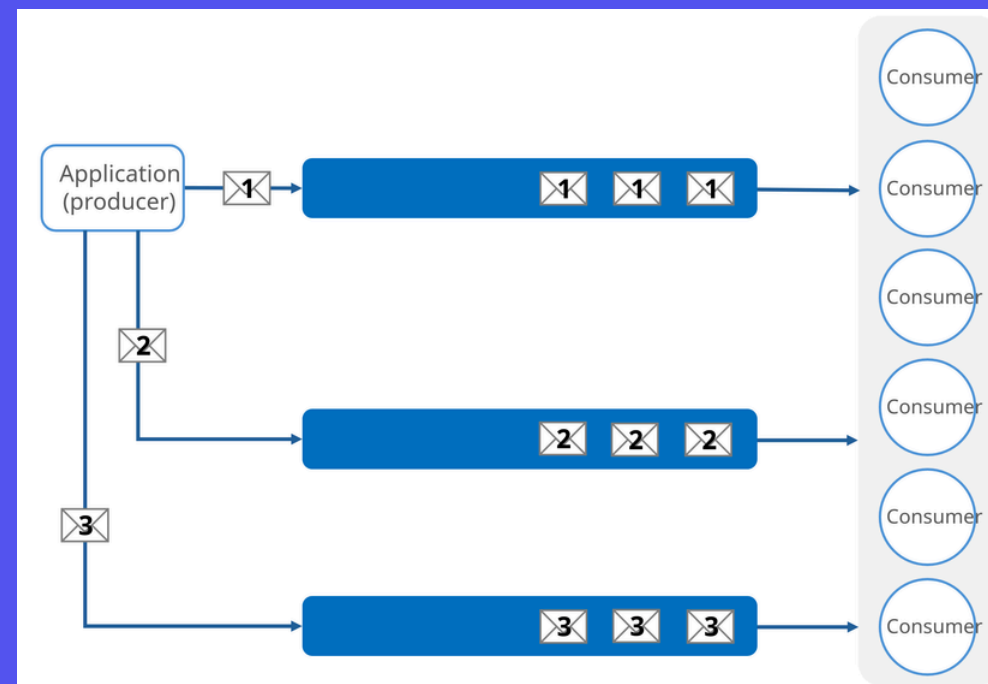
Ln 47, Col 59 Spaces: 4

Multiple Queues

Each message is assigned a priority value, and different-priority messages use separate queues.

Single consumer pool

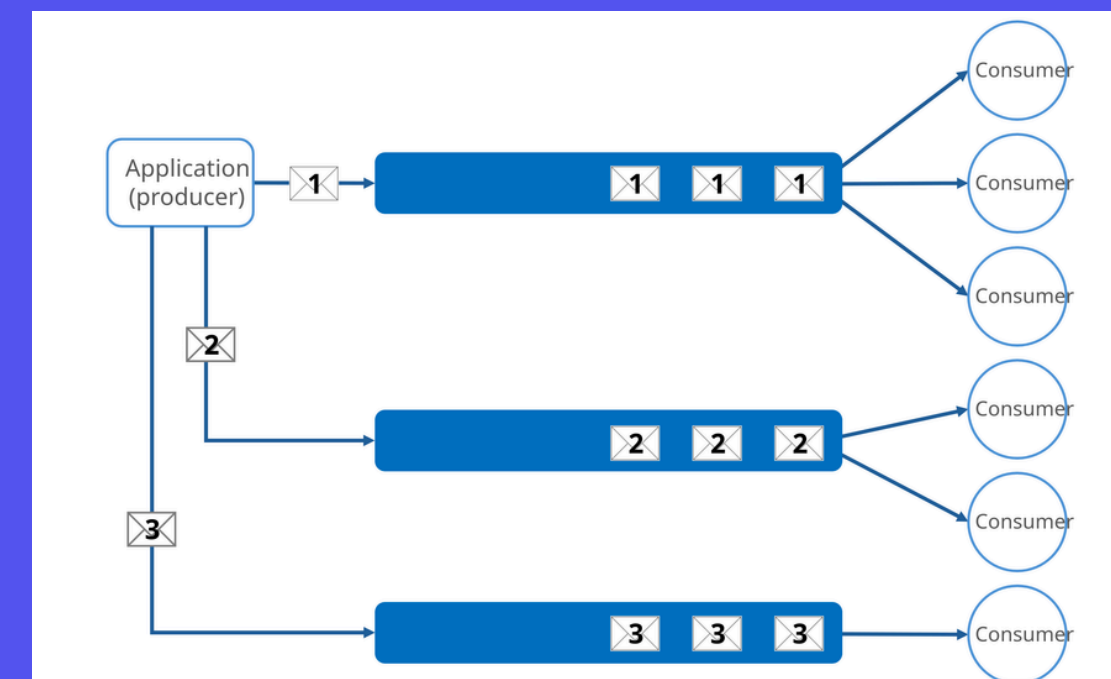
In a single pool setup, all queues share the same consumer pool.



<https://learn.microsoft.com/en-us/azure/architecture/patterns/priority-queue>

Multiple consumer pools

In a multiple consumer pool, each queue has a dedicated consumer pool.



<https://learn.microsoft.com/en-us/azure/architecture/patterns/priority-queue>

Multiple Queue Example

```
1 from collections import deque
2 import time
3
4 class MultiplePriorityQueue:
5     def __init__(self):
6         self._queues = {
7             1: deque(),
8             2: deque(),
9             3: deque()
10        }
11
12    def send(self, priority: int, content: str):
13        """Envía un mensaje a la cola correspondiente a su prioridad."""
14        if priority not in self._queues:
15            raise ValueError("Prioridad no soportada")
16        self._queues[priority].append(content)
17        print(f"[SEND] Mensaje '{content}' añadido a cola de prioridad {priority}.")
18
19    def receive(self) -> tuple[int, str] | None:
20        """
21        Recibe un mensaje, siempre priorizando las colas de mayor prioridad.
22        Primero revisa cola 1, luego 2, luego 3.
23        """
24        for priority in [1, 2, 3]:
25            if self._queues[priority]:
26                content = self._queues[priority].popleft()
27                return priority, content
28        return None
29
30 if __name__ == "__main__":
31     queue = MultiplePriorityQueue()
32     queue.send(3, "Tarea baja")
33     queue.send(1, "Tarea URGENTE")
34     queue.send(2, "Tarea normal")
35     queue.send(1, "Otra URGENTE")
36
37     print("\n--- Procesando mensajes (Multiple Queue) ---")
38     while True:
39         item = queue.receive()
40         if item is None:
41             break
42         priority, content = item
43         print(f"[PROCESS] Procesando: '{content}' (Prioridad {priority})")
44         time.sleep(0.5)
```

```
OUTPUT  DEBUG CONSOLE  PROBLEMS  TERMINAL  PORTS
[Running] python -u "/home/noodle/Documents/Plataformas II/Priority_Queue_Pattern_Example/Single_Queue/Multiple_Queue.py"
```

AWS

Ln 36, Col

When to use this pattern

- Different work needs different speed (e.g., premium vs. standard users).
- Traffic spikes happen – process critical tasks first, delay the rest.



When to avoid it

- All tasks are equally important – FIFO works fine.
- Tasks depend on each other across priorities – reordering breaks things.



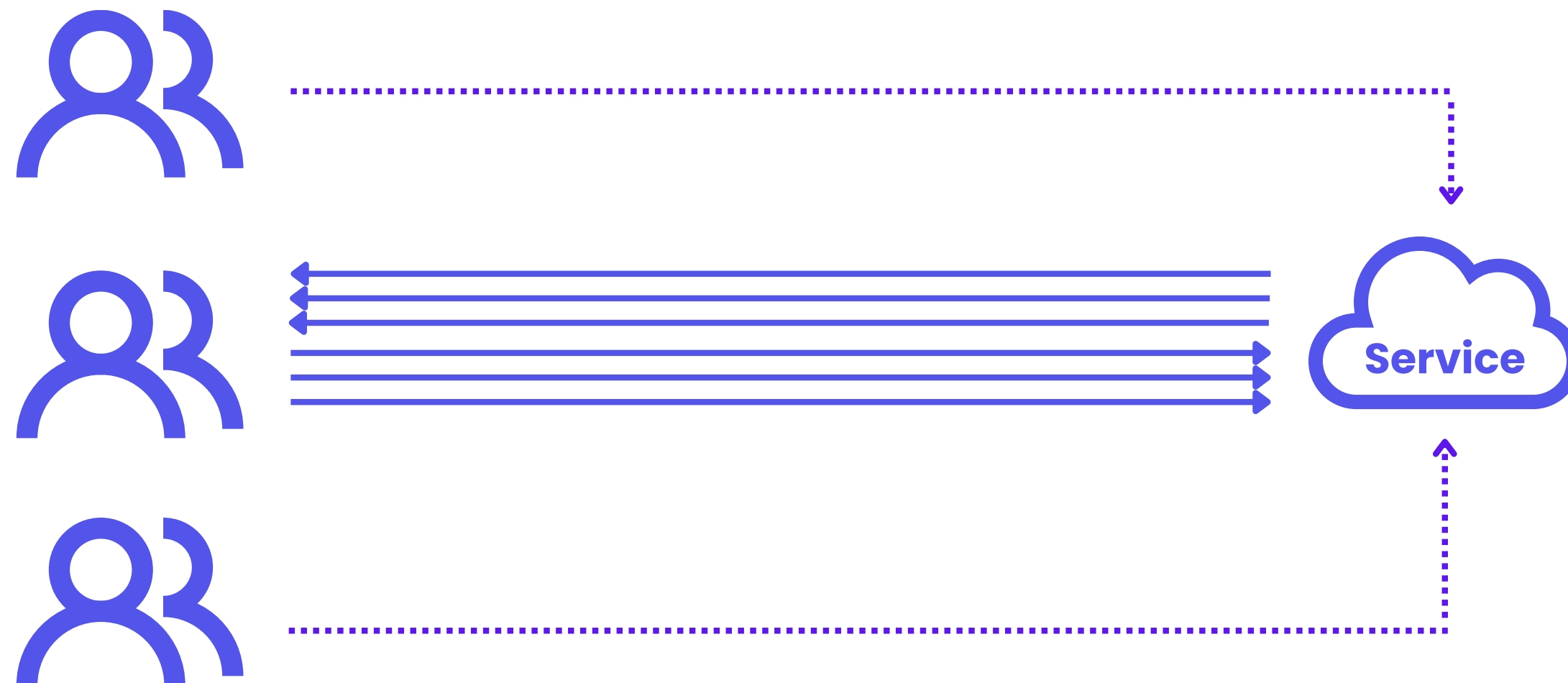
Recommendations

- Set clear priorities – define levels and allocate resources.
- Scale & monitor – adjust consumers, track latency, isolate bad messages.
- Serve high-priority first – use load leveling to handle spikes.
- Control costs – scale down off-peak, watch queue fees.



Throttling

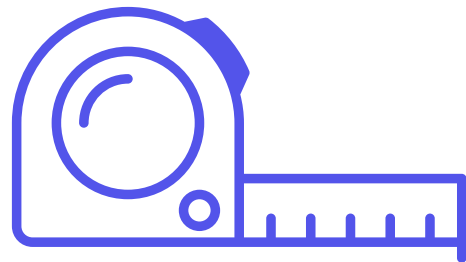
Control the consumption of resources used by an instance of an application, an individual tenant, or an entire service.



Problems and Considerations

Architect & Measure

Implement throttling at the saturation point, choose the right algorithm, and decide between local or centralized Redis counters.



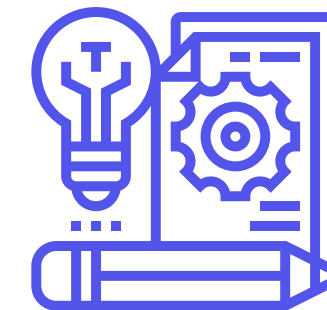
React & Signal

Reject early on p99 latency, return 429/503 with Retry-After, and propagate back-pressure upstream to prevent retry storms.



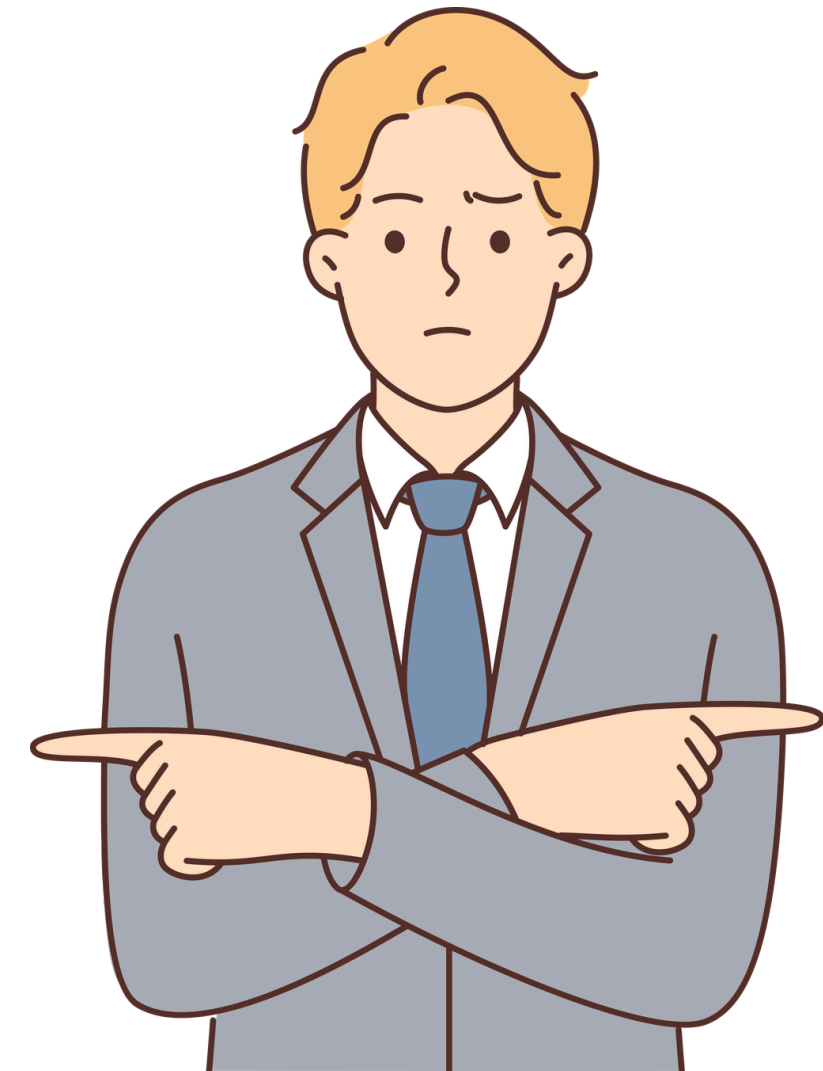
Adapt & Evolve

Make limits runtime-configurable, normalize costs by operation, and revisit them as workloads change.

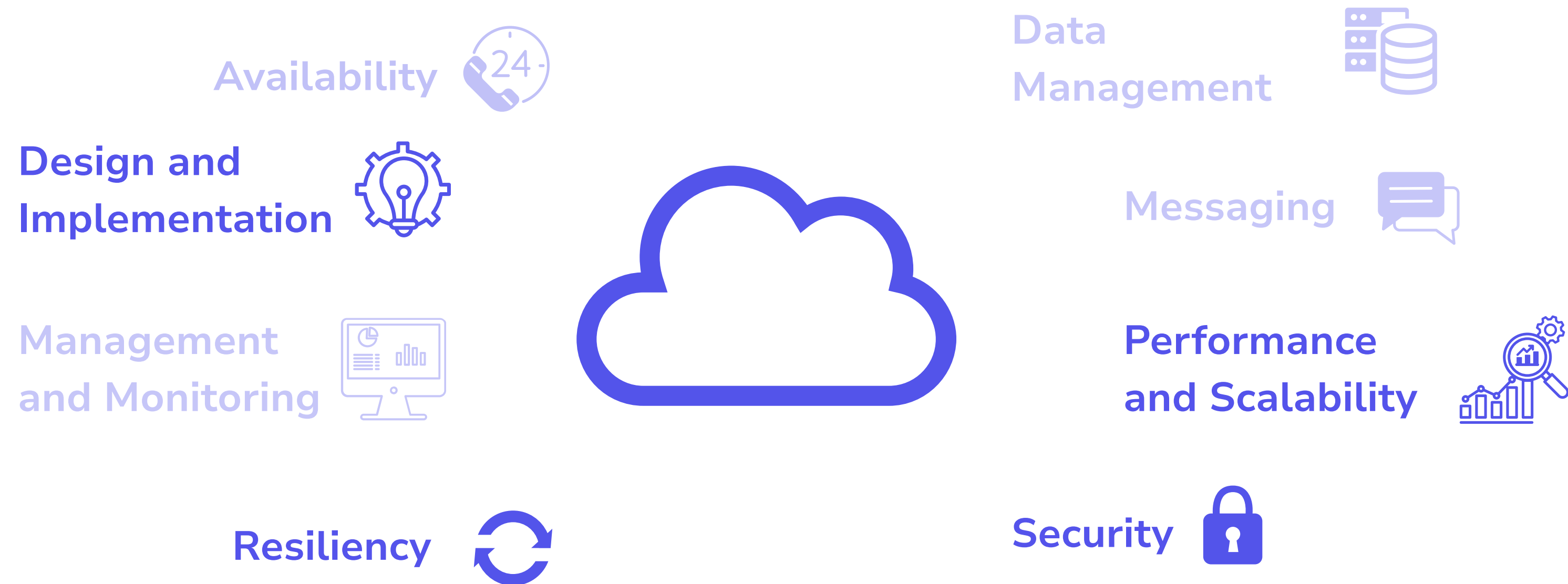


When to use this pattern

- To keep a system within its SLOs.
- To prevent a single tenant from monopolizing application resources.
- To handle bursts in activity.
- To limit the maximum resource level that a system needs.
- To reduce low-value compute during periods of high grid carbon intensity.



Workload design



Throttling Example

```
1 import time
2
3 class Throttler:
4     """
5     Tracks requests per tenant within a fixed time window (1 second).
6     Resets counters automatically when the window expires.
7     """
8     def __init__(self, window_seconds=1):
9         self.window = window_seconds
10        self.counters = {}
11        self.window_start = time.time()
12
13    def allow_request(self, tenant_name, limit):
14        """
15        Check if a request from this tenant is allowed.
16        'limit' is the maximum number of requests allowed per window.
17        Returns True if allowed, False otherwise.
18        """
19        now = time.time()
20        if now - self.window_start >= self.window:
21            self.counters.clear()
22            self.window_start = now
23            print("--- Counter reset for new window ---")
24
25        if tenant_name not in self.counters:
26            self.counters[tenant_name] = 0
27
28        if self.counters[tenant_name] >= limit:
29            return False
30
31        self.counters[tenant_name] += 1
32        return True
33
34 def demo():
35     print("=== THROTTLING PATTERN DEMONSTRATION ===")
36     print("Two tenants with different limits:")
37     print("  - Premium: 5 requests/second")
38     print("  - Free: 2 requests/second")
39     print("The demo will run 10 simulated seconds.\n")
40
41     throttler = Throttler(window_seconds=1)
42
43     tenants = [
44         ("Premium", 5),
45         ("Free", 2)
46     ]
47
48     import random
49
50     requests = []
51     for _ in range(20):
52         tenant = random.choice(tenants)
53         delay = round(random.uniform(0.1, 0.3), 2)
54         requests.append((tenant[0], delay))
55
56     requests.extend([
57         ("Premium", 0.05),
58         ("Premium", 0.05),
59         ("Premium", 0.05),
60         ("Premium", 0.05),
61         ("Premium", 0.05),
62         ("Free", 0.05),
63         ("Free", 0.05),
64         ("Free", 0.05),
65     ])
66
67     print("Starting request simulation...\n")
68     time.sleep(1)
69
70     for tenant_name, delay in requests:
71
72         limit = next(t[1] for t in tenants if t[0] == tenant_name)
73
74         allowed = throttler.allow_request(tenant_name, limit)
75
76         status = "ACCEPTED" if allowed else "REJECTED"
77         print(f"[{time.strftime('%H:%M:%S')}] {tenant_name:>10} : {status}")
78
79         time.sleep(delay)
80
81     print("\n=== DEMO FINISHED ===")
82
83 if __name__ == "__main__":
84     demo()
```

OUTPUT DEBUG CONSOLE PROBLEMS TERMINAL PORTS

[Running] python -u "/home/noodle/Documents/Plataformas II/Priority_Queue_Pattern_Example/Throttling/Throttler.py"

AWS

Thanks